Technical report: Beneficia

Offline drafts of Swiss beneficial owner declarations, built with Apertus.

- **Track:** Track 2B, Own Project
- **Event:** Online
- **Team:** Beneficia (solo)
- **Demo:** https://youtu.be/cDgRgegBRk0

1. Summary

Since 1 October 2026, every Swiss company must report its beneficial owners to the federal transparency register (TJPG / LTPM, SR 955.3; ordinance TJPV / OTPM, AS 2026 364). Fiduciaries prepare these reports for many clients from confidential documents in German, French or Italian, often scanned. Those documents are covered by professional secrecy and must not be sent to a cloud service.

Beneficia is an air-gapped assistant that turns a client's document folder into a sourced draft declaration. Apertus v1.5 8B runs locally on CPU (GGUF, llama.cpp) and only extracts facts, each with the id of the line it comes from. Deterministic, unit-tested Python checks every citation, builds the ownership graph and applies the law. Anything the text does not settle goes to a review list. The tool never files anything.

Headline result: on 14 fictitious cases in German, French and Italian, the final pipeline runs in a Docker container with **no network interface** and identifies exactly the expected beneficial owners and review points in **all 14 cases**, including a degraded scan read by OCR. It produced **zero silent errors**: no wrong beneficial owner was ever marked as settled, in any run. **100 %** of the references shown resolve to a source line, and holdings extraction reaches 98 % precision and 98 % recall. A case of 1 to 3 short documents takes 1 to 7.5 minutes on a 6-core desktop CPU without a GPU.

2. Architecture

```
Docker container, no network at runtime
  web UI (FastAPI, DE/FR) or CLI (analyze, eval)
    1 ingest     PDF text layer, Tesseract OCR (deu, fra, ita), DOCX, TXT -> numbered lines D1.P1.L7
    2 extract    8 narrow passes -> llama-server on 127.0.0.1, Apertus v1.5 8B Q4_K_M, JSON schema grammar
    3 resolve    every name and number checked on its cited line, otherwise dropped
    4 validate   totals, sums, duplicates
    5 rules      TJPG art. 4, TJPV art. 1 to 4, 12, 13, 15
    6 report     findings, review list, documents to request, note, SVG diagram, JSON
  data/ volume: case folders and results stay on the host
```

Target architecture (mandatory)

b) Air-gapped. It also fits a) on premise.

- **Build time.** Base image, OS packages, Python wheels and the model file are downloaded. The model is pinned by revision and SHA256.
- **Run time.**
  - The model server listens on 127.0.0.1 inside the container. The web app is published on the host's 127.0.0.1 only.
  - The application has no outbound client code.
  - `make offline-test` starts the container with `--network none` and runs the full evaluation.
  - It fails if any network interface other than loopback exists, if an outbound connection succeeds, if a case produced no result, or if a citation does not resolve.
- **External dependencies at run time:** none.

The template's `LLM_BASE_URL` can point to a Swiss sovereign endpoint serving Apertus (architecture c). That configuration was not tested.

3. Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-8B`, text-only conversion, GGUF Q4_K_M (5.06 GB).
- **How it is used:** inference, for structured information extraction in German, French and Italian.
- **Where it runs:** local weights, llama.cpp server (official image `server-b10548`, pinned by digest), CPU, inside the container.

Weights and access.

- **Lineage.** The weights come from `swiss-ai/Apertus-v1.5-8B` (gated "auto": users accept the Apertus acceptable use policy) through `andreasmartin/apertus-v1.5-8b-text`. That conversion removes the image and audio tokenizers and maps the decoder to llama.cpp's existing `apertus` architecture.
- **File used.** `Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF`, revision `248ec68`, sha256 `a037df8d…f299`, Apache 2.0.
- **Access.** That repository is not gated, so a judge can build the image with no Hugging Face account. The Apertus acceptable use policy still applies to its use.
- **Consequence.** Apertus 1.5 is multimodal, but this GGUF is text only. Scanned pages therefore go through Tesseract OCR before Apertus.

Serving flags (both are required).

- `--override-kv tokenizer.ggml.eos_token_id=int:68`: Apertus ends a turn with `<|assistant_end|>` (id 68), while the converted file declares `</s>` (id 2). Without the override, generation does not stop.
- `--jinja`: applies the model's own chat template.
- Reasoning ("deliberation") is never enabled, so no thinking tokens are spent.

Prompting.

- Each document is cut into windows of 60 numbered lines.
- Eight passes each ask one narrow question:
  - companies;
  - total capital, read only from capital lines;
  - holdings in tables;
  - holdings written as sentences (one invented example, wrapped lines joined);
  - a retry on register-like lines that produced no holding;
  - control agreements;
  - board roles;
  - person identification.
- Every row ends with a line id. The JSON schema given to llama.cpp limits that id to the ids present in the window, so the model cannot cite a line it was not shown.
- Passes run only on windows whose text contains relevant words (for example "Aktien", "parts", "azioni").

What Apertus does not do.

- It never computes a share, a threshold or a conclusion.
- Its numbers are copied values. Code re-checks them against the cited line.

Why Apertus fits.

- A Swiss model with strong German, French and Italian coverage.
- Open weights, so it can run where the documents are.
- Its licence allows on-premise deployment.

4. Data

- **No real data.** All companies, persons, addresses and figures are invented. Every document carries a "fictitious" banner.
- **The evaluation set.**
  - 14 cases with 20 documents.
  - Formats: text PDF, TXT, DOCX and a degraded JPEG scan.

  - Languages: 7 German, 5 French and 2 Italian cases.
  - Each case has an `expected.json`: the expected holdings, beneficial owners (basis, band, status), review reasons and missing documents.
- **Generation.** `data/tools/make_cases.py` (reportlab, python-docx, Pillow) produces the files reproducibly.
- **Coverage.** The set contains:
  - direct holdings and a person just under 25 percent;
  - indirect control through one and two intermediaries, and a "multiplication trap" (40 percent of a 100 percent holding is not control);
  - a voting agreement and a fiduciary holding;
  - nobody above 25 percent (default manager);
  - inconsistent figures and an undocumented foreign holder;
  - a DOCX table, a percentage written in prose, a wrapped sentence and a poor scan.
- **Licence:** CC BY 4.0, also stated per instance in `expected.json`. Published on Hugging Face from the HackApertus template: https://huggingface.co/datasets/Chinorab/beneficia-cases.
- **Legal sources.** The TJPG final text (Federal Assembly, BBl 2025 2900) and the TJPV (AS 2026 364, DE and FR) were read from fedlex.admin.ch and parlament.ch. Each coded rule cites its article in code and in the output.

5. Evaluation

Metrics. Computed by `beneficia.evaluate` against `expected.json`:

- *holdings precision / recall*, matched on normalised holder, company and quantity;
- *owners correct*: exact set of beneficial owners with basis, band and status;
- *review recall*: every expected review reason raised;
- *silent errors*: a settled finding that is wrong or not expected (the target is zero);
- *citation validity*: share of displayed references that resolve to a document line;
- *time per case*, with the hardware stated.

Baseline. A first full design used one large extraction schema with verbatim quotes. On one-page excerpts it took 511 to 1151 s per document on the development CPU and confused fields. The table below compares the first working pipeline (v1, run stopped after 6 complete cases), two intermediate runs, the final code run natively, and the same final code run in the Docker image with `--network none` (`make offline-test`, the reference). Only the code changed between runs: same cases, same model file, temperature 0.

Metric	v1	v2	v3	final native	final, air-gapped container
Cases	6	14	14	14	14
Cases fully correct (owners and review)	1 / 6	6 / 14	7 / 14	13 / 14	14 / 14
Beneficial owners exactly right	1 / 6	6 / 14	8 / 14	13 / 14	14 / 14
Expected review reasons all raised	6 / 6	12 / 14	11 / 14	14 / 14	14 / 14
Silent errors (settled and wrong)	0	0	0	0	0
Citation validity	100 %	100 %	100 %	100 %	100 %
Holdings precision (all)	84 %	86 %	95 %	98 %	98 %
Holdings recall (all)	76 %	79 %	81 %	94 %	98 %
Holdings recall (DE)	82 %	83 %	83 %	92 %	100 %
Holdings recall (FR)	67 %	71 %	71 %	94 %	94 %
Holdings recall (IT)	75 %	83 %	100 %	100 %	100 %
Seconds per case, mean	124.9	110.5	148.5	174.3	246.2
Seconds per case, max	148.7	226.8	238.8	316.8	449.4

Reading these numbers.

- *Tuned on the same cases.* Every error was diagnosed on these 14 cases and fixed in code, so the final figures describe a pipeline tuned on the cases it is scored on. A held-out set, ideally real anonymised folders, is the next step (section 8). The two properties that do not depend on tuning held in every run: zero silent errors and 100 % citation validity.
- *Native and container.* The scan (case12) fails natively, because Tesseract is installed only in the image. In the container it passes (OCR confidence 93). case14 has one wrong holding in both runs. The checks routed it to review, and the owner is still correct.
- *Times.* All times were measured on a desktop after hours of continuous load, so generation fell from about 5 to about 4 tokens per second. The container runs inside the WSL2 virtual machine of Docker Desktop on Windows, which made it about 40 % slower than the native build here. A Linux host would avoid that layer.
- *Replay.* `make replay` recomputes every score from the recorded model outputs in `data/results/`, in seconds and without the model.

Error analysis. `docs/evaluation.md` lists each iteration, the failure it removed, and examples of model errors caught by the checks.

Role of other models. No other model is used, neither at run time nor for evaluation. Scoring is fully deterministic against hand-written expected answers.

6. Limitations

- **Small set.** 14 fictitious cases, written by the developer. Real share registers are messier (multi-page tables, handwritten annotations, several share classes).
- **CPU speed.** 4 to 5 tokens per second on a 6-core desktop CPU. A case of 1 to 3 short documents took 98 to 317 s natively and 74 to 449 s in the container. Time grows linearly with the number of pages, and each document needs about 6 model calls. A GPU or a larger CPU would cut this; long folders are a batch job, not an interactive one.
- **Duplicate merging.** Two identical lots held by the same person in the same company are merged into one, to avoid double counting the same row read by two passes.
- **Not decided automatically.**
  - Control by other means, concert, trusts, foundations, associations, SICAV (TJPG art. 5), listed and pension holders are flagged only.
  - Mixing direct and indirect holdings to reach 25 percent is flagged, because the text does not settle it.
- **Identification.** Data is taken only from the documents. Missing fields are listed as documents to request and are never filled in.
- **OCR.** Scan quality depends on Tesseract. Pages under 70 mean confidence turn findings into "to be reviewed".
- **Model artefact.** The GGUF is a community conversion with no published perplexity. Quality relative to the original weights was not measured.

7. Reproducibility

- **Code:** https://github.com/Chinorab/beneficia, tag `hackapertus-submission` (the commit that was submitted).
- **Build:** `make run` builds the image (model download about 5 GB) and starts the interface.
- **Numbers:** `make eval` reproduces the evaluation, and `make offline-test` proves the air gap.
- **Determinism:** temperature 0 and the llama.cpp grammar. Small numeric differences across CPUs are possible.
- **Hardware used for the reported numbers:** Intel Core i5-8400 (6 cores, AVX2), 32 GB RAM, no GPU, Windows 11.
  - Reference run: Docker Desktop 29.8 on WSL2 (6 CPUs, 16.7 GB), image built from a clean clone, `--network none`.
  - Native runs: llama.cpp b10549 with the same model file and flags.
- **Recorded outputs:** `data/results/` holds the reference run. Inside the image, `make replay` reproduces 14/14 in seconds without the model. Natively it gives 13/14, because the scan needs OCR.

8. Next steps

- Real anonymised case folders from partner fiduciaries, with their consent, to measure what matters.
- Share classes with different voting rights, SICAV and trust rules (TJPV art. 5 to 9).
- A reviewer workflow: accept or reject each extracted fact, then re-run the rules instantly.
- An export in the field order of the EasyGov form (still filed by the fiduciary).
- GPU or quantised KV cache settings for faster batches, and the 70B model on a Swiss sovereign cloud for hard cases.

License

Creative Commons Attribution 4.0 (CC BY 4.0). Code under Apache 2.0. All HackApertus projects are open source.

References

- TJPG, final vote text, 26 September 2025, BBl 2025 2900 (parlament.ch, Curia Vista 24.046).
- TJPV, Verordnung vom 12. Juni 2026, AS 2026 364 (fedlex.admin.ch), German and French.
- Federal Office of Justice, transparency register, transpareg.admin.ch.
- swiss-ai/Apertus-v1.5-8B, Hugging Face. katya228/Apertus-v1.5-70B-text-GGUF model card (conversion method and EOS flag).
- llama.cpp, ggml-org, server image `server-b10548`.